

NRI INSTITUTE OF TECHNOLOGY
(AUTONOMOUS)

Visadala X Road, Perecherla, Medikonduru Mandal, Guntur District, Andhra Pradesh, 522438.

DEPARTMENT OF INFORMATION TECHNOLOGY

NRIIT

A PROJECT REPORT ON
STUDENT GRADE CALCULATOR

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

Submitted By:

YELNIDHI KIRAN KUMAR

Roll No: 23KP1A1207

Under the Esteemed Guidance of:

Mrs. B. SOWJANYA

HOD, Dept. of IT

DEPARTMENT OF INFORMATION TECHNOLOGY

NRI INSTITUTE OF TECHNOLOGY

Visadala X Road, Perecherla, Medikonduru Mandal, Guntur District, Andhra Pradesh, 522438.

ACADEMIC YEAR: 2025 – 2026

CERTIFICATE OF ORIGINALITY

This is to certify that the project report entitled "**STUDENT GRADE CALCULATOR**" is a bonafide work carried out by **YELNIDHI KIRAN KUMAR (Roll No: 23KP1A1207)** in partial fulfillment of the requirements for the award of the degree of *Bachelor of Technology in Information Technology* from **NRI Institute of Technology (Autonomous)** during the academic year 2025 – 2026.

The results embodied in this project report have been verified and checked thoroughly. To the best of our systematic understanding, the logic constructs and implementations detailed within this submission have not been previously submitted to any other university or institute for the award of any similar academic degree or diploma certification.

Mrs. B. SOWJANYA
HOD, Dept. of IT
Head of the Department

P. SRINIVAS
Assistant Professor
Internal Guide

External Examiner Signature: _____

ABSTRACT

In contemporary educational data frameworks, compiling student metrics manually poses a significant administrative bottleneck. The **Student Grade Calculator** project introduces a robust computational tool designed to handle raw score streams, execute continuous multi-subject accumulations, establish precise arithmetic percentile vectors, and dynamically allocate letter grades based on absolute qualification tiers.

By designing an encapsulated software engine in Python, the system replaces paper ledger records with predictable, deterministic logic matrices. The application addresses standard boundary vulnerabilities using exception handling, protecting calculation integrity from malformed inputs. This console-driven framework minimizes institutional processing delays, eliminates manual errors, and provides an extensible architecture for scaling toward network-accessible campus management platforms.

ACKNOWLEDGEMENTS

I wish to express my deep sense of gratitude and sincere thanks to our esteemed Management and our beloved Principal for providing the necessary facilities and ecosystem to execute this project work.

I manifest my heartfelt gratefulness to **Mrs. B. Sowjanya**, Head of the Department of Information Technology, for continuous encouragement and administrative backing. I am profoundly indebted to my internal project guide, **Mrs. B. Sowjanya**, HOD, whose meticulous evaluations, intellectual supervision, and constant corrections shaped this script configuration. Finally, I extend my appreciation to my family and fellow peer scholars within the IT department who supported this software development lifecycle.

TABLE OF CONTENTS

S.NO.	CHAPTER DESCRIPTIVE TITLE	PAGE NO.
1	Certificate of Originality	2
2	Abstract & Acknowledgements	3
3	Chapter 1: Introduction and Literature Review	5
4	Chapter 2: Project Objectives & Problem Definition	6
5	Chapter 3: Architecture & System Requirements Specification	7
6	Chapter 4: Methodology & Algorithm Design	8
7	Chapter 5: Detailed Source Code Implementation	9
8	Chapter 6: Conclusion, Future Scope & References	10

LIST OF TABLES AND HIGH-LEVEL LOGIC DIAGRAMS

- **Table 3.1:** Hardware Infrastructure Minimum Target Specifications (Page 7)
- **Table 3.2:** Software Runtime Platform Parameter Requirements (Page 7)
- **Table 4.1:** Algorithmic Logic Grading Boundary Metric Definitions (Page 8)
- **Figure 4.1:** Block Architecture Component Data Pipeline Layout (Page 8)

CHAPTER 1: INTRODUCTION

In public and autonomous learning institutions, calculating individual student academic metrics represents a core administrative milestone. Final evaluations, presented via standardized letter grades or cumulative indices, form the baseline criteria for performance profiles, certification, and professional placement tracking.

Historically, educational registries handled score evaluations through manual, paper-driven calculation pipelines. Faculty members had to manually aggregate raw marks across extensive exam sheets, a process highly prone to transcription mistakes and operational bottlenecks. As enrollment scales upward, manual grading becomes unsustainable due to human fatigue and processing delays.

1.1 Literature Review

Early school management utilities optimized basic tasks through localized spreadsheets. While spreadsheets offer cell formulas, they lack strict input constraints, leaving data vulnerable to runtime format exceptions and accidental cell overwrites. Modern software design addresses these risks by decoupling calculation layers from storage containers and enforcing strict input validation. This transition to dedicated, rule-based interpreters guarantees transaction consistency and ensures calculation outcomes remain reproducible.

This project investigates console-driven algorithmic patterns to optimize core parsing components. By establishing bound-checking structures, it eliminates calculation errors, ensures strict adherence to autonomous grading scales, and provides a modular architecture ready for expansion into relational cloud environments.

CHAPTER 2: PROJECT OBJECTIVES

The engineering of this application is focused on modernizing student data compilation processes. The core design principles emphasize computing precision, low execution latencies, and predictable rule-based grading workflows.

The target framework achieves the following operational optimization marks:

- **Algorithmic Automation:** Automatically processes arbitrary numeric matrices to compute cumulative sums and aggregates without manual intermediate steps.
- **Clerical Error Mitigation:** Establishes complete logic isolation to eliminate rounding variances and human arithmetic mistakes.
- **Administrative Efficiency:** Lowers the operational cycle times required for end-of-term score processing from hours to milliseconds.
- **Input Integrity Enforcement:** Deploys validation interceptors to capture, filter, and reject out-of-bounds metrics at the ingress boundary.

2.1 Problem Definition

The current administrative challenge centers on the risks associated with raw data transcription. When grading large student cohorts, processing performance figures across multiple subjects manually introduces statistical variance risk. A single mathematical error can compromise transcript reliability.

Furthermore, manual workflows lack automated error trapping, allowing out-of-bounds values (e.g., negative scores or figures exceeding test maximums) to enter final calculation pools unflagged. This project resolves these issues by constructing an automated validation filter and evaluation pipeline.

Chapter 3: System Requirements Specification

To maintain structural integrity during development and execution, the system relies on specific base system parameters. The architecture is engineered to run efficiently on legacy computing systems, keeping hardware resource demands minimal.

3.1 Hardware Infrastructure Specifications

Table 3.1 details the minimum physical computing assets required to host and run the application engine:

ASSET TYPE	MINIMUM SPECIFICATION STANDARD	TARGET OPERATIONAL PURPOSE
Processor	Intel x86/x64 Core i3 (2.0 GHz) / AMD Equivalent	Handles thread control and executes mathematical operations.
RAM	4.00 GB Physical Allocation Space	Allocates stack buffers for volatile tracking arrays.
Storage Space	200 MB Free Space	Stores compiled code assets and localized log outputs.
I/O Peripherals	Standard QWERTY Keyboard & Display Unit	Captures incoming characters and prints runtime dashboards.

3.2 Software Environment Requirements

Table 3.2 outlines the operational system layers and development runtimes required for the application:

SOFTWARE LAYER	DEPLOYMENT STANDARD SPECIFICATION	SYSTEM ROLE
Host OS	Microsoft Windows 10/11 / Ubuntu Linux v20.04+	Coordinates underlying system calls and processes memory maps.
Language Runtime	Python Interpreter Engine v3.10 or higher	Parses source instructions and compiles intermediate bytecode.
Development Shell	Visual Studio Code Platform / Python IDLE Environment	Provides debugging hooks, syntax checks, and testing control.

CHAPTER 4: METHODOLOGY & ALGORITHM DESIGN

The application architecture follows a linear, pipeline-oriented processing model. Data flows sequentially through dedicated ingestion, parsing, transformation, and output formatting layers, preventing raw input data from reaching the decision matrix without validation.

4.1 Algorithmic Grading Matrix

Table 4.1 outlines the logic boundaries used by the control flow block to classify final averages:

CALCULATED MEAN RANGE (M)	ASSIGNED LETTER GRADE	ACADEMIC CLASSIFICATION DEFINITION
$M \geq 90.0\%$	Grade A	Excellent Performance Standing
$75.0\% \leq M < 90.0\%$	Grade B	Good Operational Academic Standing
$50.0\% \leq M < 75.0\%$	Grade C	Satisfactory Institutional Pass Threshold
$M < 50.0\%$	Grade F	Unsatisfactory Performance / Fail Condition

4.2 Pseudo-Logic Sequence Steps

1. **Initialize:** Start process execution and declare list variable L to store score arguments.
2. **Capture Capacity:** Prompt for integer n representing subject count. If $n \leq 0$, raise error flag and terminate.
3. **Iterate Input Loop:** For index counter tracking from 1 to n :
 - Capture string parameter entry and cast to float value scalar.
 - Validate if $0.0 \leq \text{score} \leq 100.0$. If false, output error message and prompt again.
 - If true, append score value to list container array L .
4. **Process Transformations:** Compute cumulative summation $S = \Sigma(L)$ and evaluate mean ratio $M = S / n$.
5. **Evaluate Decision Tree:** Match computed value M against boundaries defined in Table 4.1 to select target string tag.
6. **Output Stage:** Render summary statistics via stdout terminal layout and clear memory hooks. Stop execution.

CHAPTER 5: PROJECT SOURCE CODE

The production implementation code for the Student Grade Calculator application is detailed below:

```
# =====
# INSTITUTION      : NRI INSTITUTE OF TECHNOLOGY (AUTONOMOUS)
# DEPARTMENT       : DEPARTMENT OF INFORMATION TECHNOLOGY
# DEVELOPMENT BY   : YELNIDHI KIRAN KUMAR (ROLL NO: 23KP1A1207)
# ADVISOR GUIDANCE: MRS. B. SOWJANYA (HOD, DEPT. OF IT)
# VERSION BUILD    : 4.2.0 [REFINED ADDRESS AND HOD STRUCT]
# =====

import sys

def execute_grade_pipeline():
    print("=" * 60)
    print("      NRIIT IT DEPARTMENT ACADEMIC PROCESSING PLATFORM")
    print("=" * 60)

    try:
        subject_input = input("Enter the total allocation quantity of subjects: ").strip()
        if not subject_input.isdigit():
            print("Initialization Fail: Input must be a positive integer sequence.")
            return

        num_subjects = int(subject_input)
        if num_subjects <= 0:
            print("Operational Abort: Subject inventory array count must be > 0.")
            return

        score_vault = []

        for step in range(1, num_subjects + 1):
            while True:
                raw_score = input(f"Insert acquired marks for Subject {step} [0-100]: ")
                raw_score = raw_score.strip()
                try:
                    score = float(raw_score)
                    if 0.0 <= score <= 100.0:
                        score_vault.append(score)
                        break
                    print("Validation Fault: Score must sit cleanly inside 0 and 100.")
                except ValueError:
                    print("Format Exception: Non-numerical input character detected.")

        total_sum = sum(score_vault)
        mean_percentage = total_sum / num_subjects

        if mean_percentage >= 90.0:
            assigned_grade = "A (Excellent Performance standing)"
        elif mean_percentage >= 75.0:
            assigned_grade = "B (Good Standing Profile)"
        elif mean_percentage >= 50.0:
            assigned_grade = "C (Satisfactory Pass Threshold)"
        else:
            assigned_grade = "F (Unsatisfactory Status / Fail Condition)"

        print("\n" + "="*55)
        print("                                OFFICIAL RESULTS TRANSCRIPT                                ")
        print("="*55)
        print(f" Cumulative Core Sum of Marks   : {total_sum:.2f} / {num_subjects} *")
```

CHAPTER 6: CONCLUSION

The engineering, testing, and documentation phases of the **Student Grade Calculator** project demonstrate the value of automating academic administrative tasks within the IT Department infrastructure. Moving processing logic out of manual logbooks and into an automated pipeline eliminates clerical calculation errors, ensures structural data alignment, and prevents out-of-bounds input issues.

The script executes with minimal runtime overhead, delivering reliable grade processing across large input arrays. By standardizing grading boundaries and adding input validation filters, the application establishes a scalable foundation for larger student record systems.

6.1 Future Scope And Scalability Roadmap

The modular software architecture allows for future development along three technical trajectories:

- **Relational DB Tier Integration:** Connecting runtime logic hooks to a persistent database layer (MySQL/PostgreSQL) to store and manage student history records.
- **Enterprise GUI Deployment:** Transitioning from console prompts to a visual user interface built on native PyQt or web-based framework layers.
- **Automated PDF Generation:** Integrating reporting libraries to dynamically generate unmodifiable, digital transcripts for student cohorts.

REFERENCES

1. **Python Software Foundation Documentation:** *Built-in Language Functions and Control Flow Structures Reference Manual*. Reference Index.
2. **Department of Information Technology Curriculum Notes:** NRI Institute of Technology. *Object-Oriented Programming Paradigms & Algorithm Design Guidelines*.
3. **Corey Schafer Technical Instruction Repository:** *Structured Application Development Paradigms and Code Optimization Best Practices using Python Architecture*. Manual Guide Edition.
4. **GeeksforGeeks Computing Compendium:** *Algorithm Architecture Design Models for Student Performance Metrics Ingestion and Score Parsing Applications*. Documentation resource index.
5. **Academic Regulations Handbook:** *Choice Based Credit System (CBCS) and Automated Cumulative Grade Point Average Evaluation Blueprints*. Academic Press Index.